Course ID: SW 302
User Interface (UI) Development

Lecture#3
Introduction to HTML

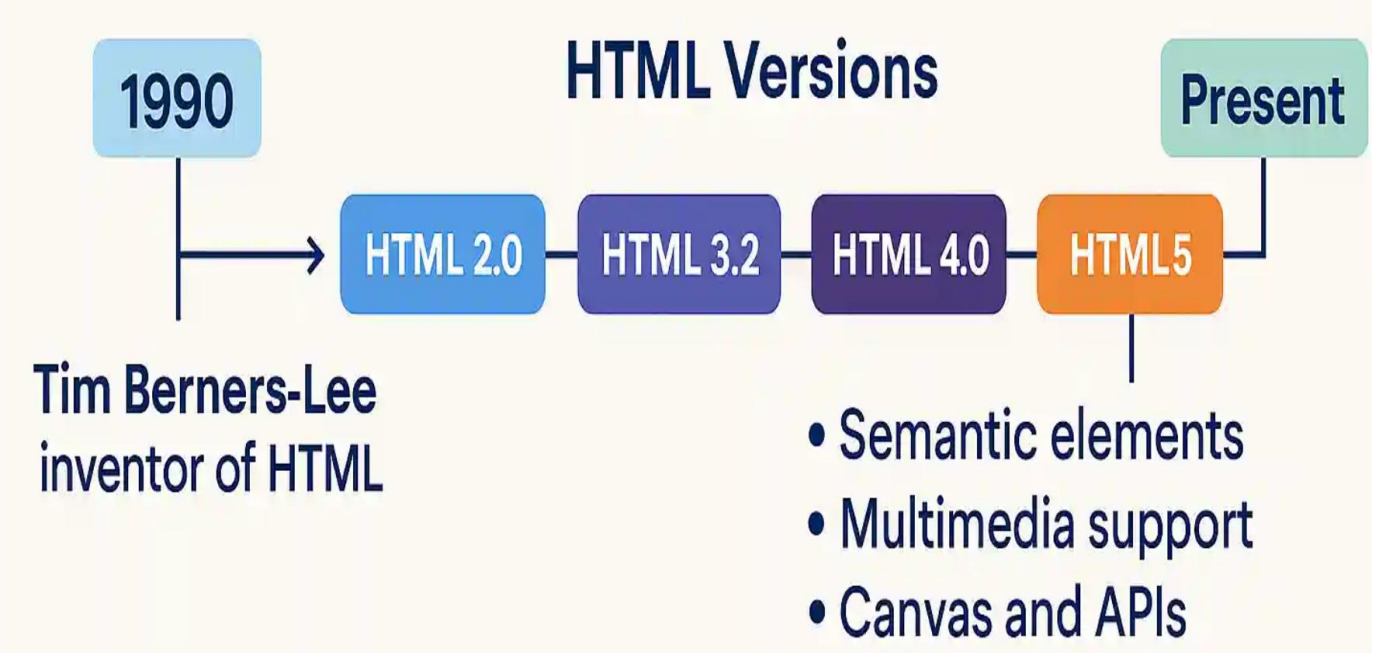
Lecture Outline

- Cont: What is HTML5?
- HTML5 Built-in Validation

Cont: What is HTML5?

HTML Evolution

HTML5 was officially finalized in 2014 and continues to evolve. Developed by **W3C (World Wide Web Consortium).**



Overview - Key Features of HTML5

Semantic Elements – new tags that give meaning to content:

<header>, <footer>, <article>, <section>, <nav>, <aside>

Improves accessibility & SEO.

Multimedia Support – no plugins needed:

<audio> and <video> to play media directly in the browser.

Supports controls, autoplay, captions, etc.

Graphics & Animation::

<canvas> → for 2D graphics, games, data visualization.

<svg> → scalable vector graphics.

Overview - Key Features of HTML5

Form Enhancements:

New input types like **email, url, date, range, color.**

Attributes like required, placeholder, and autofocus. Improves accessibility & SEO.

APIs for Rich Web Apps:

Geolocation API (get user's location)

Local Storage & Session Storage (store data on the client side).

WebSockets (real-time communication).

Cross-Platform & Mobile Friendly / Designed for responsive design

HTML5 Doctype

The DOCTYPE declaration tells the browser **which version of HTML** to expect, so it can render the page correctly.

In older versions **(HTML4, XHTML)**, the DOCTYPE was very long and strict, for example:

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"**>**

HTML5 simplified this into just one short line:

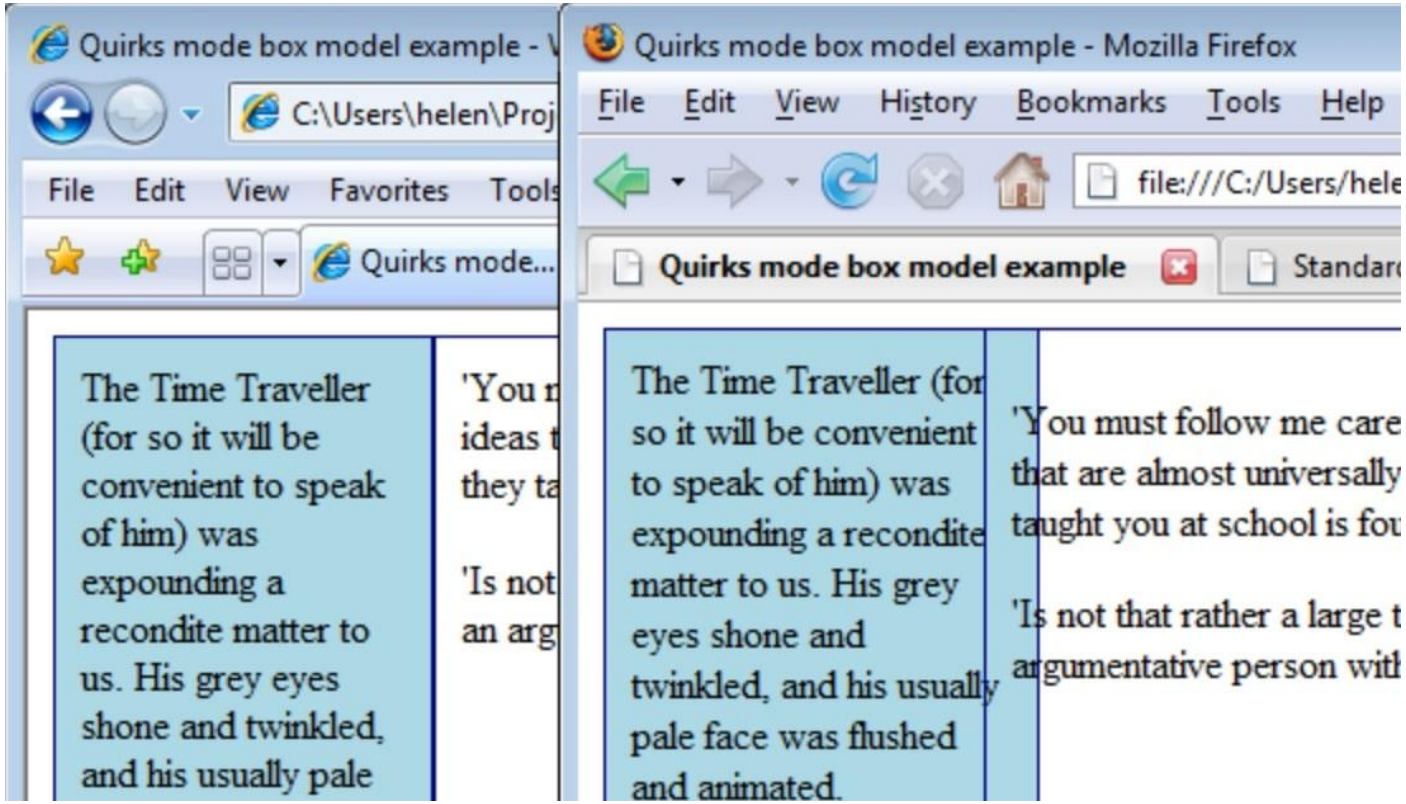
<!DOCTYPE html>

Tells the browser: ***"This is an HTML5 document."***, Much simpler, easier to remember!

Why is Doctype Important?

If you don’t use DOCTYPE, browsers may go into **strange mode**, where they try to guess how to render the page (often breaking layouts).

If a user doesn’t enter a **doctype** during the creation of the **HTML** web page or email template, then there is no guarantee that it will open up for your website visitors as intended, **especially across multiple browsers**



New Semantic Elements in HTML5

Why Semantic Elements?

- Provide **meaningful structure** to web pages (beyond just ** **).
- Improve **readability**, **SEO**, and **accessibility** (for screen readers).
- Easier for developers to understand page layout.

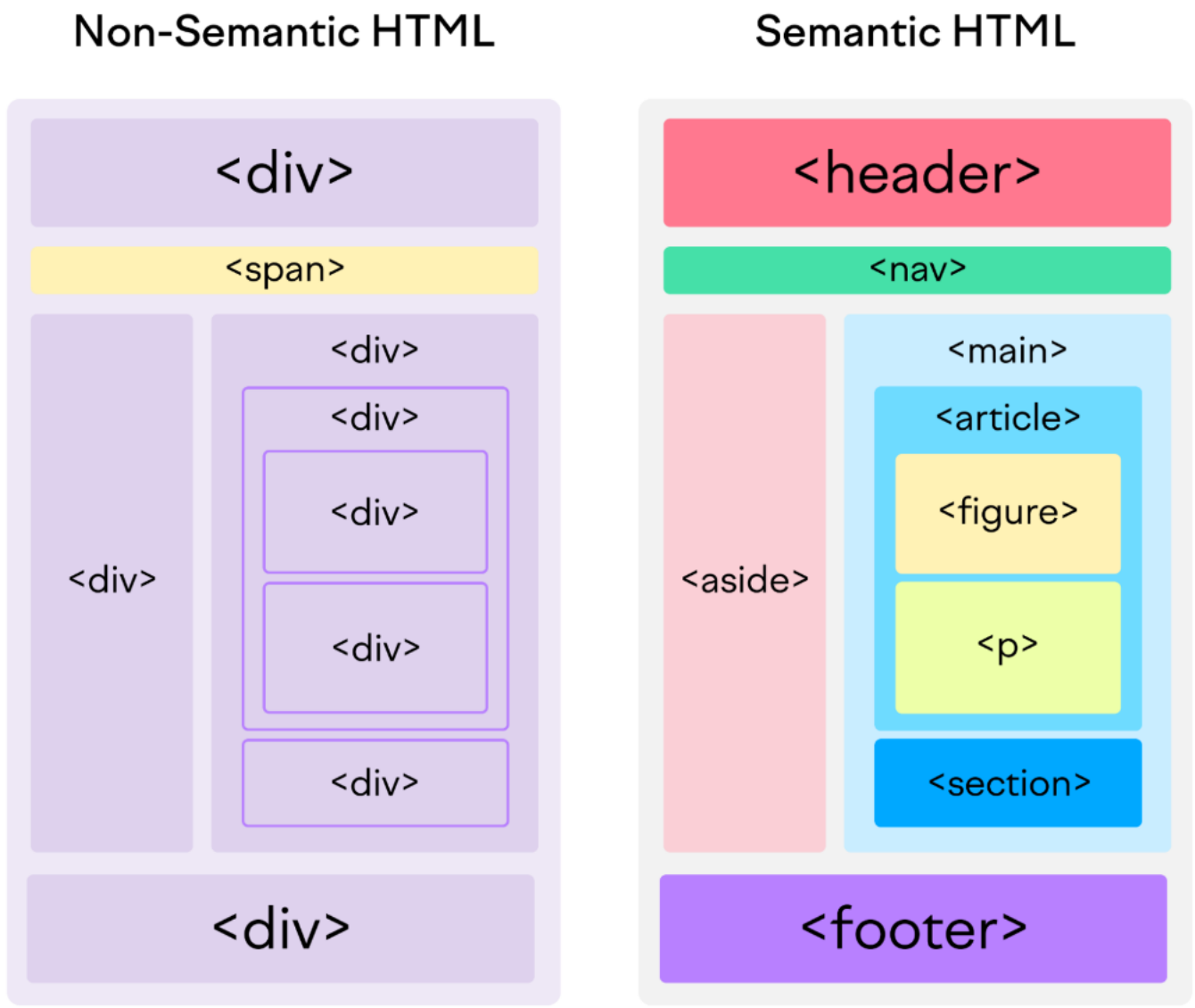
Element	Meaning / Use	Example
<header>	Introductory content, logo, nav, or heading	Website top banner
<footer>	Closing content, copyright, contact info	Bottom of page
<section>	Thematic grouping of content	Chapter in an article
<article>	Self-contained, independent content	Blog post, news article
<nav>	Major navigation links	Menu, navbar
<aside>	Side content related to main content	Sidebar, ads, tips
<main>	Main unique content of the page	Excludes headers/footers/nav

New Semantic Elements in HTML5

```
<body>
 <header>
  <h1>My Blog</h1>
  <nav>
   <a href="#">Home</a> | <a href="#">Posts</a>
  </nav>
 </header>

 <main>
  <article>
   <h2>HTML5 Basics</h2>
   <p>HTML5 introduces new semantic elements...</p>
  </article>

  <aside>
   <p>Related Links</p>
  </aside>
 </main>
 <footer>
  <p>&copy; 2025 My Blog</p>
 </footer>
</body>
```



New Semantic Elements: <main> </main>

- Identifies the primary content of a page or application
- Helps users with screen readers get to the main content of the page

```
<body>
<header>…</header>
<main>
  <h1>Humanist Sans Serif</h1>
  …content continues…
</main>
</body>
```

New Semantic Elements: Headers and Footers

- **Header:** identifies the introductory material that comes at the beginning of a page, section, or article (logo, title, navigation, etc.).
- **Footer:** indicates the type of information that comes at the end of a page, section, or article (author, copyright, etc.)


```
<article>
  <header>
    <h1>More about WOFF</h1>
    <p>by Jennifer Robbins, <timedatetime="2017-11-11">
      November 11, 2017</time></p>
  </header>
  <!-- ARTICLE CONTENT HERE -->

  <footer>
    <p><small>Copyright &copy;2017 Jennifer
Robbins.</small></p>
    <nav>
    <ul>
      <li><a href="/">Previous</a></li>
      <li><a href="/">Next</a></li>
    </ul>
    </nav>
  </footer>
</article>
```

New Semantic Elements: <section> </section>

- **Section** identifies a thematic section of a page or an article. It can be used to divide up a whole page or a single article:

```
<section>
  <h2>Typography Books</h2>
  <ul>
     <li>…</li>
  </ul>
</section>

<section>
  <h2>Online Tutorials</h2>
  <p>These are the best tutorials on the Web.</p>
  <ul>
    <li>…</li>
  </ul>
</section>
```

New Semantic Elements: <article> </article>

- **Article** is used for self-contained works that could stand alone or be used in a different context (such as syndication).
- Useful for magazine/newspaper articles, blog posts, comments, etc.

```
<article>
  <h1>Get to Know Helvetica</h1>
  <section>
    <h2>History of Helvetica</h2>
    <p>…</p>
  </section>

  <section>
    <h2>Helvetica Today</h2>
    <p>…</p>
  </section>
</article>
```

New Semantic Elements: <aside> </aside>

- **Aside** identifies content that is separate from but tangentially related to the surrounding content (think of it as a sidebar).

```
<h1>Web Typography</h1>
<p>Back in 1997, there were competing font formats and tools
for making them…</p>
<p>We now have a number of methods for using beautiful fonts
on web pages…</p>
<aside>
  <h2>Web Font Resources</h2>
  <ul>
  <li><a href="http://typekit.com/">Typekit</a></li>
  <li><a href="http://fonts.google.com">Google Fonts</a></li>
  </ul>
</aside>
```

New Semantic Elements: <nav> </nav>

- **nav** identifies the primary navigation for a site, a lengthy section, or an article. It provides more semantic meaning than a simple unordered list.

```
<nav>
<ul>
  <li><a href="/">Serif</a></li>
  <li><a href="/">Sans-serif</a></li>
  <li><a href="/">Script</a></li>
  <li><a href="/">Display</a></li>
  <li><a href="/">Dingbats</a></li>
</ul>
</nav>
```

Do HTML5 semantic tags like <header>, <nav>, <aside>, <main>, and <article> have **default positioning or layout behavior**?

Default Positioning / Layout Behavior for Semantics Tags?

No Default Position or Layout. The tags are primarily **structural and semantic,** meaning they help describe the purpose and meaning of the content to browsers, assistive technologies (like screen readers), and search engines, but **they don't control visual placement** on the page.


```
<header>Header</header>
<nav>Navigation</nav>
<main>Main
content</main>
<aside>Sidebar</aside>
<footer>Footer</footer>
```

In this example will appear **stacked vertically**, one after another , not laid out like a typical webpage (with nav on top, sidebar to the side, etc.)

Multimedia Support in HTML5

Before HTML5, Web developers relied on **Flash, Silverlight**, or third-party plugins to embed audio & video.

Problems:

- Required extra installations.
- Security risks.
- Not supported on all devices (e.g., iPhones didn't support Flash).

After HTML5, Native multimedia support - no plugins required.

- Built-in tags:
  - <audio> → for music, sound effects, podcasts.
  - <video> → for movies, tutorials, streaming.
- Browser controls: play, pause, volume, fullscreen.

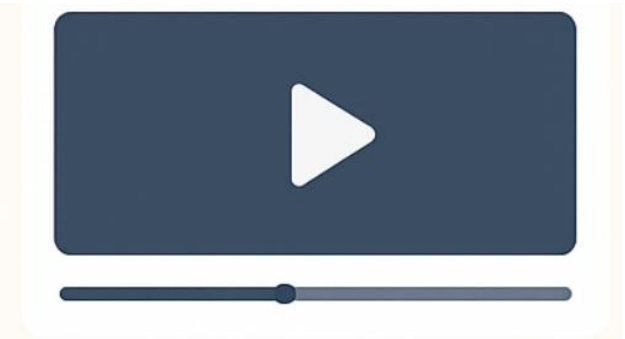
Advantages

- No need for external plugins
- Cross-platform, mobile-friendly
- Supports multiple formats
- JavaScript API for custom controls

Multimedia Support in HTML5

Video Example:

```
<video width="400" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
  Your browser does not support the video tag.
</video>
```



Audio Example:

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>
```

Advantages

- No need for external plugins
- Cross-platform, mobile-friendly
- Supports multiple formats
- JavaScript API for custom controls

Forms in HTML5

What Changed?

HTML5 introduces new input types and attributes to make forms smarter, easier, and more user-friendly.

New Input Types

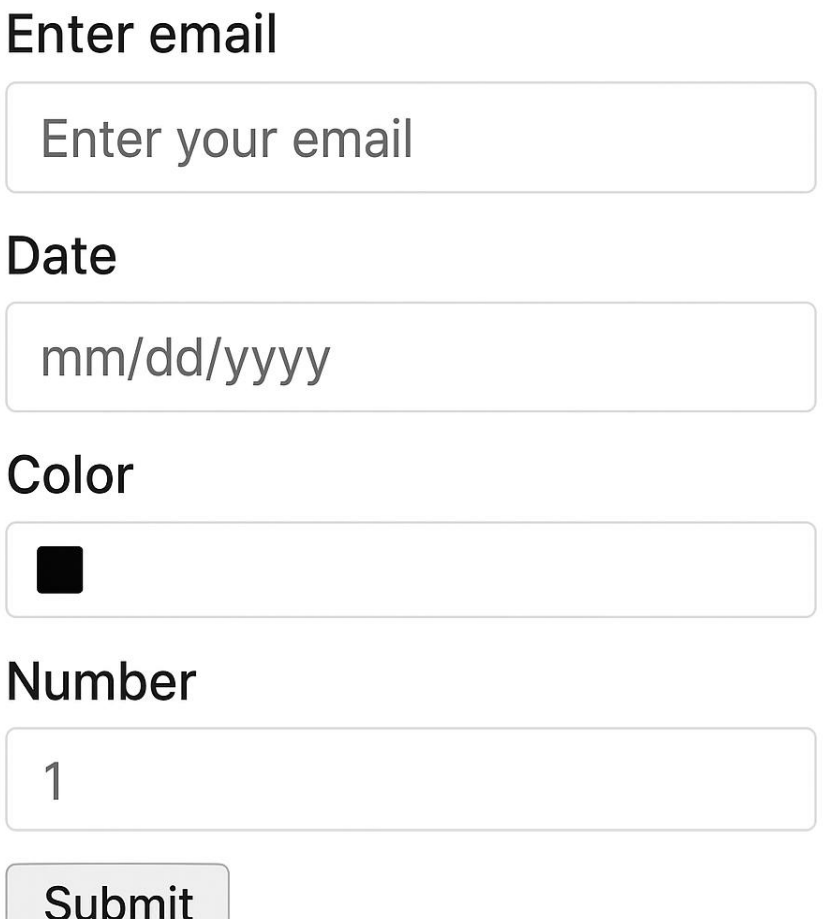
- **email** → validates email format.
- **url** → checks for a valid website address.
- **number** → allows min/max values.
- **range** → slider input.
- date, time, datetime-local, month, week → date/time pickers.
- **color** → color selector.

New Attributes

- **placeholder** → shows hint text
- **required** → field must be filled
- **pattern** → regex validation
- **autofocus** → cursor auto-focuses
- **autocomplete** → suggests stored values
- **min, max, step** → set constraints

Forms in HTML5

```
<form>
  <input type="email" placeholder="Enter email" required>
  <input type="date">
  <input type="color">
  <input type="number" min="1" max="10">
  <input type="submit" value="Submit">
</form>
```



Benefits

- Built-in **validation** (less JavaScript needed).
- **Mobile-friendly** controls.
- Better **user experience** with interactive inputs.

“If **HTML** is not a programming language, then how can it perform built-in validation in forms**?**”

“If HTML is not a programming language, then how can it perform built-in validation in forms?”

HTML itself is not a programming language — it’s a markup language. But **HTML5** defines rules for the browser about how certain elements should behave. **The browser’s engine (which is programmed) enforces those rules.** When you use attributes like:

- **type="email"** → must be a valid email format.
- **required** → cannot leave the field empty.

The browser automatically checks before form submission

How **built-in validation** works:

```
<form>
  <input type="email" required>
  <input type="submit">
</form>
```

- `type="email"` tells the browser: *“This field must contain a valid email format (something@domain.com)”*.
- `required` tells the browser: *“Don’t allow submitting this form if the field is empty.”*

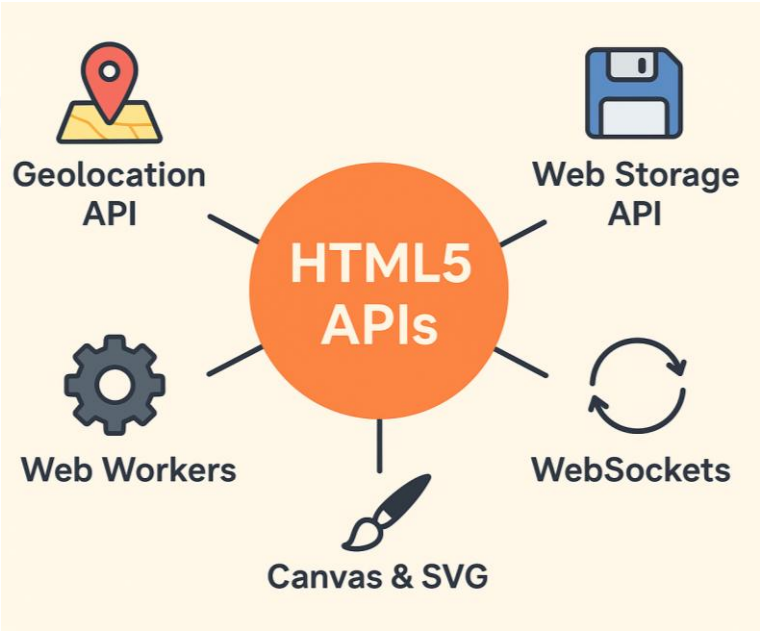
When you click **Submit**, the **browser** checks the field. If it’s empty or not an email, it blocks submission and shows a validation message.

HTML5 APIs

What are APIs in HTML5?

When HTML5 was standardized, many **APIs were introduced** to work *with* JavaScript to make web apps more powerful.

- API = Application Programming Interface
- Provide extra features & functionality to web pages.
- Allow developers to build rich, interactive web apps (beyond just static HTML).
- **APIs** are accessed through JavaScript.
- **HTML5** just provides the hooks and environment for these APIs.
- **JavaScript** = The "glue" that actually uses those APIs



- 🌍 Geolocation
- 💾 Web Storage
- ⚙️ Web Workers
- 🔄 WebSockets
- 🎨 Canvas & SVG

HTML5 APIs – Example #1

Geolocation API: Get user's current location (with permission).
HTML5 just provides the **hooks and environment** for these APIs.

```
<button onclick="getLocation()">Get Location</button>

<script>
  function getLocation() {
    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(pos => {
        alert("Latitude: " + pos.coords.latitude);
      });
    } else {
      alert("Geolocation not supported");
    }
  }
</script>
```

The `<button>` is **HTML5**.

The **Geolocation API** is used **via JavaScript**.

HTML5 APIs – Example #2

Local Storage (persistent even after closing browser)

```
<button onclick="toggleTheme()">Toggle Dark Mode</button>
<script>
  // Load saved theme
  if (localStorage.getItem("theme") === "dark") {
    document.body.style.background = "black";
    document.body.style.color = "white";
  }

  function toggleTheme() {
    if (localStorage.getItem("theme") === "dark") {
      localStorage.setItem("theme", "light");
      document.body.style.background = "white";
      document.body.style.color = "black";
    } else {
      localStorage.setItem("theme", "dark");
      document.body.style.background = "black";
      document.body.style.color = "white";
    }
  }
</script>
```

The `<button>` is **HTML5**.

Here, the theme preference is **saved in Local Storage**.

Even if the user **closes and reopens** the browser, it **remembers** dark mode.

“**HTML5** is not just a markup language; it’s the backbone of the interactive, multimedia-rich, and cross-platform web we experience today!.”

Accessibility where HTML's native semantics are not enough!

ARIA (Accessible Rich Internet Applications) Labels

ARIA labels: It's a set of attributes that you can add to HTML elements to make web content more accessible to users with disabilities, particularly those using screen readers or assistive technologies.

What does it do?

Its provides a **text label** that describes an element to screen readers when there isn't already visible text or when the visible text isn't descriptive enough.

ARIA **is not technically part of HTML5**, but it is closely integrated with HTML5 and designed to work alongside it.

ARIA Example#1

Example 1: Button with only an icon

```
<button aria-label="Search">
  <img src="search-icon.png" alt="">
</button>
```

Visually, the user only sees a magnifying-glass icon.
The screen reader will announce: **“Search, button”, because of aria-label**.

Without the **aria-label**, the button would be announced as “button” or “image”, which isn’t meaningful.

ARIA Example#1

Example 2: Input field without a visible label

```
<input type="text" aria-label="Email address">
```

The screen reader will read this as “Email address, edit text”, so even though no <label> is visible, users know what the field is for.

AIRA and HTML5 are complementary: HTML5 gives semantic meaning through elements like <header>, <nav>, <main>, and ARIA extends accessibility where HTML's native semantics are not enough.

AIRA +

AIRA and HTML5 are Complementary

HTML5 has many built-in semantics that are already accessibility-aware:

```
<nav>Navigation links</nav>
```

A screen reader automatically recognizes <nav> as a navigation region, **no ARIA needed.**

But if you use a non-semantic element:

```
<div role="navigation“ aria-label="Main menu" >Navigation links</div>
```

You can use **ARIA’s attributes** to tell assistive tech what it represents

Thank You…